TRƯỜNG: **ĐH CT TP.HCM** KHOA: **CÔNG NGHỆ THÔNG TIN** BỘ MÔN: **KHDL** MH: **TH HỌC MÁY**	**BUỔI 5. GIẢM CHIỀU DỮ LIỆU (PCA-LDA)**	

A. MỤC TIÊU

- Xác định được phương pháp giảm chiều phù hợp bộ dữ liệu.
- Đánh giá thực trạng bộ dữ liệu.
- Cài đặt các thuật toán giảm chiều.
- Kết hợp các thuật toán phân lớp, phân cụm cho bộ dữ liệu cụ thể trước và sau giảm chiều.
- Đánh giá được hiệu quả của quá trình sử dụng thuật toán giảm chiều thông qua thực nghiệm.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1. Sử dụng Google Colab để viết chương trình:

B1. Truy cập địa chỉ: https://colab.research.google.com/

B2. Vào file → new notebook



B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

2. PCA (Principal Component Analysis)

Một thuật toán giảm chiều dữ liệu quan trọng trong học máy và thống kê. PCA giúp giảm số lượng đặc trưng trong dữ liệu xuống số chiều thấp hơn trong khi vẫn giữ được phần lớn

thông tin của dữ liệu. Việc giảm chiều này giúp dễ dàng trực quan hóa, tăng tốc độ tính toán và giảm độ phức tạp của mô hình, đặc biệt là trong các bài toán với dữ liệu có số chiều lớn.

PCA hoạt động thông qua việc tìm các thành phần chính (principal components) của dữ liệu. Các thành phần này là các phương của dữ liệu với phương sai cao nhất. Cụ thể, các bước trong PCA bao gồm:

- Chuẩn hóa dữ liệu: PCA nhạy cảm với các đơn vị và thang đo khác nhau của dữ liệu. Vì thế, trước tiên, dữ liệu cần được chuẩn hóa để mỗi đặc trưng có giá trị trung bình là 0 và độ lệch chuẩn là 1.
- Tính ma trận hiệp phương sai (Covariance Matrix): Ma trận này mô tả mối quan hệ giữa các đặc trưng của dữ liệu và giúp tìm hiểu các chiều có phương sai cao hơn, từ đó giúp phát hiện các thành phần chính.
- Tính toán các vector riêng (Eigenvectors) và giá trị riêng (Eigenvalues) của ma trận hiệp phương sai: Vector riêng xác định hướng của các thành phần chính, trong khi giá trị riêng cho biết tầm quan trọng của từng vector riêng. Các vector riêng có giá trị riêng lớn nhất là các thành phần chính.
- Chọn số lượng thành phần chính: Chọn số lượng thành phần chính dựa vào giá trị riêng. Tổng các giá trị riêng càng cao thì càng nhiều phương sai của dữ liệu được giữ lại. Người dùng có thể chọn số lượng thành phần chính sao cho phương sai giữ lại đạt mức mong muốn (thường là từ 90% đến 95%).
- Chuyển đổi dữ liệu: Biến đổi dữ liệu từ không gian ban đầu sang không gian của các thành phần chính. Mỗi điểm dữ liệu sẽ được biểu diễn bằng các thành phần chính đã chọn.

Ví dụ 1 minh họa về cách sử dụng PCA để giảm chiều dữ liệu từ tập dữ liệu Iris.

```
# Thêm các thư viện

    import numpy as np

    import matplotlib.pyplot as plt

    from sklearn.decomposition import PCA

    from sklearn.datasets import load_iris

    from sklearn.preprocessing import StandardScaler

# Tải dữ liệu

    data = load_iris()
```

```
    X = data.data
    y = data.target
# Chuẩn hóa dữ liệu
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
# Khởi tạo PCA với 2 thành phần chính
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_scaled)
# Kiểm tra tỷ lệ phương sai giữ lại của các thành phần
    print("Tỷ lệ phương sai giữ lại bởi mỗi thành phần chính:", pca.explained_variance_ratio_)
    print("Tổng tỷ lệ phương sai giữ lại:", sum(pca.explained_variance_ratio_))
# Hiển thị dữ liệu trên không gian mới với 2 chiều chính
    plt.figure(figsize=(8, 6))
    plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis')
    plt.xlabel("Thành phần chính 1")
    plt.ylabel("Thành phần chính 2")
    plt.colorbar()
    plt.title("Dữ liệu Iris sau khi giảm chiều bằng PCA")
    plt.show()
```

Ví dụ 2 minh họa về cách kết hợp PCA với SVM để giảm chiều dữ liệu trước khi áp dụng thuật toán phân lớp SVM.

```
# Thêm thư viện
```

```
        import numpy as np
        import matplotlib.pyplot as plt
        from sklearn.decomposition import PCA
        from sklearn.datasets import load_iris
        from sklearn.preprocessing import StandardScaler
        from sklearn.model_selection import train_test_split
        from sklearn.svm import SVC
        from sklearn.metrics import accuracy_score, confusion_matrix
        import seaborn as sns
# Tải tập dữ liệu Iris
        data = load_iris()
        X = data.data
        y = data.target
# Chuẩn hóa dữ liệu
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)
# Giảm chiều dữ liệu với PCA
        # Khởi tạo PCA với 2 thành phần chính
            pca = PCA(n_components=2)
            X_pca = pca.fit_transform(X_scaled)
# Kiểm tra tỷ lệ phương sai giữ lại của các thành phần
        print("Tỷ lệ phương sai giữ lại bởi mỗi thành phần chính:", pca.explained_variance_ratio_)
```

```
print("Tổng tỷ lệ phương sai giữ lại:", sum(pca.explained_variance_ratio_))

# Chia dữ liệu

X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.3, random_state=42)

# Huấn luyện mô hình SVM trên dữ liệu đã giảm chiều

svm_model = SVC(kernel='linear', C=1)

svm_model.fit(X_train, y_train)

# Dự đoán trên tập kiểm tra

y_pred = svm_model.predict(X_test)

# Sử dụng các chỉ số như độ chính xác và ma trận nhầm lẫn để đánh giá mô hình.

# Tính độ chính xác

accuracy = accuracy_score(y_test, y_pred)

print("Độ chính xác của mô hình SVM sau khi giảm chiều bằng PCA:", accuracy)

# Ma trận nhầm lẫn

conf_matrix = confusion_matrix(y_test, y_pred)

# Hiển thị ma trận nhầm lẫn

plt.figure(figsize=(6, 4))

sns.heatmap(conf_matrix, annot=True, cmap="Blues", fmt="d",

xticklabels=data.target_names, yticklabels=data.target_names)

plt.xlabel("Dự đoán")

plt.ylabel("Thực tế")

plt.title("Ma trận nhầm lẫn của mô hình SVM sau khi giảm chiều bằng PCA")
```

```
plt.show()
```

3. LDA (Linear Discriminant Analysis)

Một phương pháp giảm chiều dữ liệu sử dụng thông tin của các nhãn lớp để tìm ra các thành phần tối ưu nhất cho việc phân lớp. Khác với **PCA**, một phương pháp được xem như không giám sát chỉ tối ưu dựa trên phương sai dữ liệu, **LDA** là một kỹ thuật được xem như có giám sát, tức là nó dùng nhãn của dữ liệu để tối ưu hóa tách biệt giữa các lớp trong không gian mới.

Ý tưởng chính của LDA: là tìm một không gian có số chiều nhỏ hơn nhưng vẫn duy trì được sự tách biệt tốt nhất giữa các lớp. Nó đạt được mục tiêu này thông qua:

- Tối đa hóa khoảng cách giữa các lớp: Tăng khoảng cách giữa các trung tâm của mỗi lớp để các lớp càng phân biệt rõ ràng trong không gian mới.
- Tối thiểu hóa sự lan truyền trong mỗi lớp: Giảm phương sai của các điểm dữ liệu trong mỗi lớp để dữ liệu của mỗi lớp được nhóm lại gần nhau.

Quá trình thực hiện LDA bao gồm các bước sau:

- Tính toán vector trung bình: Tính vector trung bình tổng thể của toàn bộ dữ liệu cũng như vector trung bình của mỗi lớp.
- Tính toán ma trận tán xạ trong lớp (Within-class scatter matrix): Ma trận này đại diện cho sự phân tán của các điểm dữ liệu trong mỗi lớp quanh trung tâm của lớp đó.
- Tính toán ma trận tán xạ giữa các lớp (Between-class scatter matrix): Ma trận này đại diện cho sự phân tán của các trung tâm lớp so với trung tâm tổng thể của dữ liệu.
- Tìm các thành phần chính (discriminant components): Tính toán các vector riêng (eigenvectors) và giá trị riêng (eigenvalues) từ ma trận ngược của ma trận tán xạ trong lớp nhân với ma trận tán xạ giữa các lớp. Các vector riêng tương ứng với các giá trị riêng lớn nhất là các thành phần chính giúp tối ưu hóa phân lớp.
- Biến đổi dữ liệu sang không gian mới: Dữ liệu ban đầu được chuyển đổi vào không gian của các thành phần chính, với số chiều nhỏ hơn.

Ví dụ minh họa về cách sử dụng LDA để giảm chiều dữ liệu trên tập dữ liệu Iris kết hợp thuật toán Logistic Regression trên dữ liệu đã giảm chiều để xem hiệu quả phân loại của dữ liệu sau khi sử dụng LDA.

```
# Thêm các thư viện cần thiết

    import numpy as np

    import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Tải tập dữ liệu Iris
data = load_iris()
X = data.data
y = data.target
# Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Áp dụng LDA để giảm số chiều dữ liệu
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X_scaled, y)
# Kiểm tra tỷ lệ phương sai giữ lại của các thành phần
print("Tỷ lệ phương sai giữ lại bởi mỗi thành phần chính:", lda.explained_variance_ratio_)
# Trực quan hóa dữ liệu sau khi giảm chiều
plt.figure(figsize=(8, 6))
plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='viridis', s=50, alpha=0.7)
plt.xlabel("Thành phần phân biệt 1")
```

```
    plt.ylabel("Thành phần phân biệt 2")
    plt.title("Dữ liệu Iris sau khi giảm chiều bằng LDA")
    plt.colorbar()
    plt.show()
# Áp dụng mô hình phân loại trên dữ liệu đã giảm chiều
# Chia dữ liệu thành tập huấn luyện và kiểm tra
    X_train, X_test, y_train, y_test = train_test_split(X_lda, y, test_size=0.3, random_state=42)
# Khởi tạo và huấn luyện mô hình Logistic Regression
    model = LogisticRegression()
    model.fit(X_train, y_train)
# Dự đoán trên tập kiểm tra
    y_pred = model.predict(X_test)
# Tính độ chính xác
    accuracy = accuracy_score(y_test, y_pred)
    print("Độ chính xác của mô hình Logistic Regression sau khi giảm chiều bằng LDA:", accuracy)
```

III. Bài tập ở lớp

Bài 1. Sử dụng PCA để xác định số lượng thành phần chính cần thiết

Mô tả: Thực hiện PCA trên tập dữ liệu Wine và xác định số lượng thành phần chính cần thiết

để giữ lại ít nhất 95% phương sai của dữ liệu.

Yêu cầu:

1. Tải tập dữ liệu Wine từ sklearn.datasets.
2. Chuẩn hóa dữ liệu trước khi thực hiện PCA.
3. Tính toán tỷ lệ phương sai giữ lại (explained variance ratio) cho từng thành phần chính.
4. Vẽ biểu đồ Scree Plot và tích lũy phương sai để xác định số lượng thành phần chính cần thiết để giữ lại ít nhất 95% phương sai.

Bài 2. Giảm chiều dữ liệu với PCA

Mô tả: Sử dụng tập dữ liệu Iris để giảm số chiều từ 4 xuống còn 2 và 3, sau đó trực quan hóa dữ liệu trong không gian mới.

Yêu cầu:

1. Sử dụng PCA để giảm số chiều xuống còn 2 và 3.
2. Trực quan hóa dữ liệu sau khi giảm chiều trong không gian 2 chiều và 3 chiều.
3. So sánh tỷ lệ phương sai giữ lại giữa PCA 2D và PCA 3D.

Bài 3. Xác định số thành phần chính cần thiết với thuật toán LDA trên tập dữ liệu Wine.

Mô tả: Sử dụng LDA để xác định số thành phần cần thiết để tối ưu hóa phân biệt các lớp trên tập dữ liệu Wine.

Yêu cầu:

1. Tải tập dữ liệu Wine từ sklearn.datasets.
2. Chuẩn hóa dữ liệu trước khi áp dụng LDA.
3. Áp dụng LDA và giảm số chiều từ 13 xuống số chiều tối thiểu cần thiết để phân biệt tốt nhất các lớp.
4. Tính toán và hiển thị tỷ lệ phương sai giữ lại của các thành phần chính trong không gian mới.

IV. Bài tập về nhà

Bài 1. Kết hợp PCA và K-means để phân cụm dữ liệu

Mô tả: Sử dụng PCA để giảm chiều dữ liệu trước khi áp dụng thuật toán phân cụm K-means trên tập dữ liệu Digits.

Yêu cầu:

1. Tải tập dữ liệu Digits từ sklearn.datasets.
2. Sử dụng PCA để giảm chiều dữ liệu xuống còn 2 và trực quan hóa các cụm.
3. Sử dụng K-means để phân cụm dữ liệu sau khi giảm chiều.
4. So sánh kết quả phân cụm với nhãn thực tế của dữ liệu Digits và hiển thị độ chính xác.

Bài 2. Thuật toán LDA để cải thiện phân loại đa lớp với Logistic Regression

Mô tả: Sử dụng LDA để giảm số chiều của tập dữ liệu Digits và áp dụng Logistic Regression để phân loại các chữ số.

Yêu cầu:

1. Tải tập dữ liệu Digits từ sklearn.datasets.
2. Áp dụng LDA để giảm chiều dữ liệu xuống còn 9 thành phần.
3. Huấn luyện một mô hình Logistic Regression trên dữ liệu đã giảm chiều và kiểm tra độ chính xác của mô hình.
4. So sánh độ chính xác của mô hình với dữ liệu gốc (trước khi giảm chiều) và với dữ liệu sau khi giảm chiều bằng LDA.